

Solution Code - Module 3-Core Java - VirtualThreads

Below is the complete functional source code implementation for this assignment. All logic, validation rules, and structural requirements have been satisfied.

Source File: Q40_VirtualThreads.java

```
1 | import java.time.Duration;
2 | import java.time.Instant;
3 | import java.util.ArrayList;
4 | import java.util.List;
5 | import java.util.concurrent.atomic.AtomicInteger;
6 |
7 | public class Q40_VirtualThreads {
8 |     private static final int VIRTUAL_THREAD_COUNT = 100_000;
9 |     private static final int TRADITIONAL_THREAD_COUNT = 2_000; // Small count to avoid crashing JVM/OS
10 |
11 |     public static void main(String[] args) {
12 |         System.out.println("=== Java 21 Virtual Threads Performance Demo ===");
13 |         System.out.println("Operating System: " + System.getProperty("os.name"));
14 |         System.out.println("Java Version: " + System.getProperty("java.version"));
15 |
16 |         // 1. Virtual Threads Test
17 |         System.out.println("\n1. Launching " + VIRTUAL_THREAD_COUNT + " Virtual Threads...");
18 |         AtomicInteger virtualCounter = new AtomicInteger(0);
19 |         List<Thread> virtualThreads = new ArrayList<>(VIRTUAL_THREAD_COUNT);
20 |
21 |         Instant startVirtual = Instant.now();
22 |         for (int i = 0; i < VIRTUAL_THREAD_COUNT; i++) {
23 |             Thread vThread = Thread.startVirtualThread(() -> {
24 |                 virtualCounter.incrementAndGet();
25 |             });
26 |             virtualThreads.add(vThread);
27 |         }
28 |
29 |         // Wait for all virtual threads to finish
30 |         for (Thread vt : virtualThreads) {
31 |             try {
32 |                 vt.join();
33 |             } catch (InterruptedException e) {
34 |                 e.printStackTrace();
35 |             }
36 |         }
37 |         Instant endVirtual = Instant.now();
38 |         Duration durationVirtual = Duration.between(startVirtual, endVirtual);
39 |
40 |         System.out.println("    Completed!");
41 |         System.out.println("    Virtual Threads launched and completed: " + virtualCounter.get());
42 |         System.out.println("    Time taken: " + durationVirtual.toMillis() + " ms (" +
(durationVirtual.toMillis() / 1000.0) + " seconds)");
```

Solution Code - Module 3-Core Java - VirtualThreads

```
43 |  
44 |         // 2. Traditional Platform Threads Test  
45 |         System.out.println("\n2. Launching " + TRADITIONAL_THREAD_COUNT + " Traditional (Platform)  
Threads...");  
46 |         AtomicInteger traditionalCounter = new AtomicInteger(0);  
47 |         List<Thread> platformThreads = new ArrayList<>(TRADITIONAL_THREAD_COUNT);  
48 |  
49 |         Instant startPlatform = Instant.now();  
50 |         for (int i = 0; i < TRADITIONAL_THREAD_COUNT; i++) {  
51 |             Thread pThread = new Thread(() -> {  
52 |                 traditionalCounter.incrementAndGet();  
53 |             });  
54 |             platformThreads.add(pThread);  
55 |             pThread.start();  
56 |         }  
57 |  
58 |         // Wait for all platform threads to finish  
59 |         for (Thread pt : platformThreads) {  
60 |             try {  
61 |                 pt.join();  
62 |             } catch (InterruptedException e) {  
63 |                 e.printStackTrace();  
64 |             }  
65 |         }  
66 |         Instant endPlatform = Instant.now();  
67 |         Duration durationPlatform = Duration.between(startPlatform, endPlatform);  
68 |  
69 |         System.out.println("    Completed!");  
70 |         System.out.println("    Traditional Threads launched and completed: " + traditionalCounter.get());  
71 |         System.out.println("    Time taken: " + durationPlatform.toMillis() + " ms (" +  
(durationPlatform.toMillis() / 1000.0) + " seconds");  
72 |  
73 |         // 3. Performance Summary  
74 |         System.out.println("\n=== Performance Summary ===");  
75 |         double rateVirtual = (double) VIRTUAL_THREAD_COUNT / durationVirtual.toMillis() * 1000.0;  
76 |         double ratePlatform = (double) TRADITIONAL_THREAD_COUNT / durationPlatform.toMillis() * 1000.0;  
77 |         System.out.printf("Virtual Threads Rate: %.2f threads/second\n", rateVirtual);  
78 |         System.out.printf("Traditional Threads Rate: %.2f threads/second\n", ratePlatform);  
79 |         System.out.printf("Virtual threads are approx. %.1fx faster to create and run!\n", (rateVirtual /  
ratePlatform));  
80 |         System.out.println("Note: Launching 100,000 traditional threads would typically crash the JVM or  
cause an OutOfMemoryError due to OS stack limitations, while virtual threads ran seamlessly on the heap.");  
81 |     }  
82 | }
```